

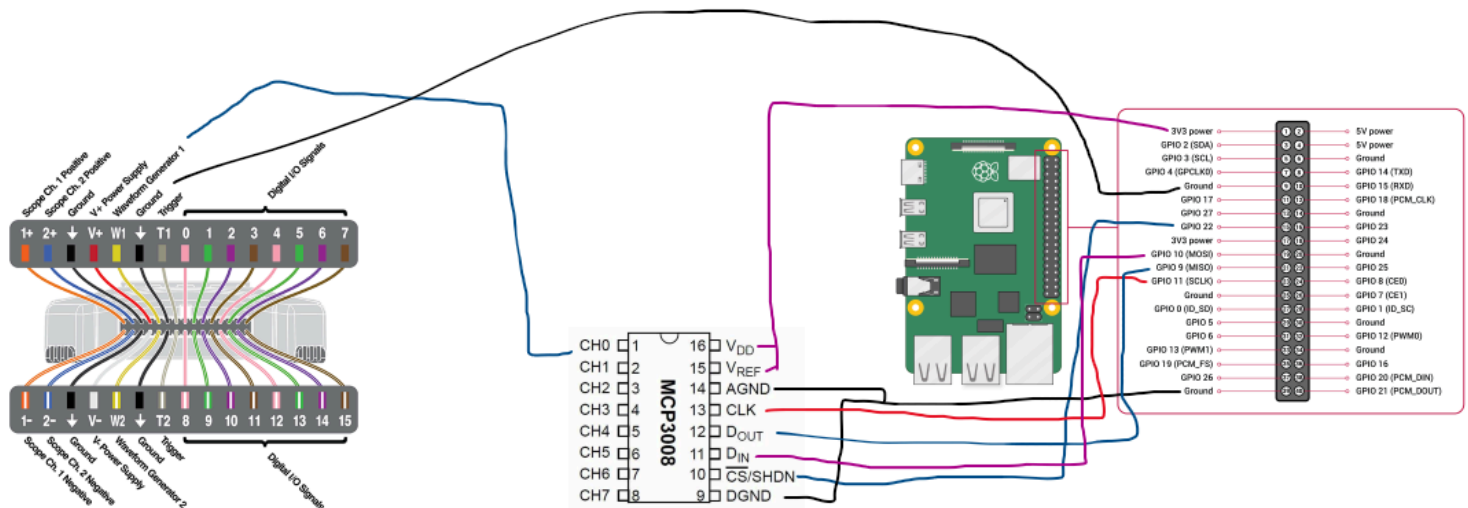
Lab 3: Interfacing Analog To Digital Converter (ADC) With A Microcontroller

Katelyn Nguyen
Oluwasegun Oluwole
CSCE 462 - 500
Group 1

20 September 2024

Hardware Description

Provide a written description of your hardware **including a wiring diagram**. Wiring diagrams can be screenshots from [multisim](#) (online or desktop applications), [circuitio.io](#), or a **LEGIBLE** photo of a hand-drawn diagram.



The hardware used in this lab consisted of a RaspberryPI 3B+, an Analog Discovery 2 Device and an ADC chip MCP3008. The raspberry pi and the ADC chip are connected via SPI as the raspberry pi serves as the master and the ADC chip as the slave. The Analog Discovery generates analog signals which are then sent to the ADC chip in channel 0, the ADC chip then converts the the analog signal to digital and sends that data to the raspberry pi via the MISO(master in slave out) wire.

Software Description

Provide a description of your code and its implementation. **Remember to include your code in your submissions as a separate file. DO NOT PROVIDE A ZIP, TAR, or other file format as your submission unless specified.**

```

22     # Initialize SPI bus
23     spi = busio.SPI(clock=board.SCK, MISO=board.MISO, MOSI=board.MOSI)
24
25     # chip select
26     cs = digitalio.DigitalInOut(board.D22)
27
28     # mcp3008 chip
29     mcp = MCP.MCP3008(spi, cs)
30
31     outPin = 22
32
33     GPIO.setmode(GPIO.BCM)
34     GPIO.setup(outPin, GPIO.OUT, initial=GPIO.LOW)
35
36     channel1 = AnalogIn(mcp, MCP.P0)

```

This part of the code is where the setup of the SPI bus, chip select, and mcp3008 chip are done, as well as the channel for reading in the Analog input in the ADC chip.

```

39     def get_data():
40         # reads a time and voltage value
41         x_points.append(time.time())
42         y_points.append(channel1.value)
43
44     def dynamic_threshold(data, factor=0.2):
45         return np.mean(np.abs(data)) * factor
46
47     def moving_avg(y_data, sma_size):
48         new_data = []
49         for i in range(len(y_data)-sma_size):
50             sum = np.sum(y_data[i:i+sma_size])
51             avg = sum/sma_size
52             new_data.append(avg)
53         return new_data
54
55     def print_graph(x_points, y_points, filename = str(int(time.time()))+".png"):
56         # fig = plt.figure()
57         plt.plot(x_points, y_points)
58         # test_shape = input("what wave shape are you testing? ")
59         print("filename: ",filename,"\n")
60         plt.savefig(filename)
61         plt.clf()

```

The function `get_data()` uses `time.time()` to get the time values for the corresponding voltages that are being read in from the ADC chip. The time values are stored in an array called `x_points` and the voltage values are stored in an array called `y_points`. The function `moving_avg` calculates a moving average of the voltages in a certain window (`sma_size`) and appends the values to an array.

This is done in order to smooth out noise in the sampled data. The function `print_graph` was used for debugging purposes and just plots the time and voltage values.

```
64  ✓ def eliminate_square_shape(first_deriv):
65      first_deriv_len = len(first_deriv)
66
67      flat_threshold = dynamic_threshold(first_deriv, factor=0.1)
68      slope_change_threshold = dynamic_threshold(first_deriv, factor=0.5)
69      # second_derivative_threshold = dynamic_threshold(second_derivatives, factor=0.5)
70
71      # Square Wave Check: Look for flat regions (near-zero slopes) and abrupt changes
72      flat_regions = np.sum(np.abs(first_deriv) < flat_threshold)
73      abrupt_changes = np.sum(np.abs(np.diff(first_deriv)) > slope_change_threshold)
74
75      if flat_regions > first_deriv_len // 3 and abrupt_changes >= 2:
76          return "Square Wave"
77
78      return "Not Square Wave"
79
```

The function `eliminate_square_shape` will take in as a parameter an array of values that represent the first derivative of the voltage values with respect to time. This function will look for flat regions and abrupt changes in the first derivative values (an impulse function) and if the number of flat regions is greater than one third of the size of the first derivative array and the number of abrupt changes in the slopes is greater than or equal to 2, then the shape is a square wave.

```

81  def get_freq(x_points, y_points_smoothed, wave_type):
82      if wave_type == "Square Wave":
83          # Square wave frequency detection by counting rising edges only
84          threshold = (max(y_points_smoothed) + min(y_points_smoothed)) / 2
85          # Detect rising edges (where the signal crosses the threshold upwards)
86          crossings = np.where((y_points_smoothed[:-1] < threshold) & (y_points_smoothed[1:] >= threshold))[0]
87          if len(crossings) < 2:
88              print("Not enough crossings to determine frequency.")
89              return None
90          crossing_times = [x_points[c] for c in crossings]
91          periods = np.diff(crossing_times)
92          if len(periods) > 0:
93              avg_period = np.mean(periods)
94              frequency = 1 / avg_period
95              print(f"Frequency = {frequency} Hz")
96              return frequency
97          else:
98              print("Not enough data to calculate frequency.")
99              return None
100      else:
101          # Use only high peaks (local maxima) for sine and triangle waves
102          high_peaks, _ = find_peaks(y_points_smoothed)
103
104          if len(high_peaks) < 2:
105              print("Not enough peaks to determine frequency.")
106              return None
107
108          peak_times = np.array([x_points[p] for p in high_peaks])
109          periods = np.diff(peak_times)
110
111          if len(periods) > 0:
112              avg_period = np.mean(periods)
113              frequency = 1 / avg_period
114              print(f"Frequency = {frequency} Hz")
115              return frequency
116          else:
117              print("Not enough data to calculate frequency.")
118              return None
119

```

This function gets the frequency of the waveform using the x points, the smoothed out y points and the wave type. If the shape is of type square wave then using the crossing times(when the waveform crosses the midpoint from high to low and low to high), the periods will be determined from the differences in those and then the average period is calculated and the frequency will be 1 over the average period. Otherwise, the shape is either a triangle or sine wave(this will be distinguished later on in the code). The difference between when the peaks occur will be stored in array called periods and the average of those periods is then calculated and then 1 over the average period will be the frequency.

```

132     while True:
133         get_data()
134         timer += 1
135
136         if timer >= window_size:
137             # smooth out the raw data using moving_avg()
138             sma_raw = moving_avg(y_points, sma_size)
139
140             # get the first derivative of the smoothed data
141             first_derivative = np.gradient(sma_raw)
142
143             # classify if the wave is square
144             wave_type = eliminate_square_shape(first_derivative)
145
146             if (wave_type == "Not Square Wave"):
147                 sma_first = moving_avg(first_derivative, sma_size)
148                 second_derivative = np.gradient(sma_first)
149                 second_wave_type = eliminate_square_shape(second_derivative)
150
151                 if (second_wave_type == "Square Wave"):
152                     wave_type = "Triangle Wave"
153                 else:
154                     wave_type = "Sine Wave"
155
156             print(f"The wave is classified as: {wave_type}")
157
158             # Get the frequency of the waveform
159             freq = get_freq(x_points[:len(sma_raw)], sma_raw, wave_type)
160             print()
161
162             if freq and freq < 5:
163                 if wave_type == "Sine Wave":
164                     window_size = 20000
165                     sma_size = 15 # decrease this to get less false sines (supposed to be triangle)
166                 else:
167                     window_size = 20000 # Increase window size for low frequencies
168                     sma_size = 18
169             elif freq and freq > 47: # fixing windows for high frequencies (specifically for improved triangle shape identification)
170                 window_size = 10000
171                 sma_size = 4
172             else:
173                 window_size = 10000 # Default window size for other frequencies
174                 sma_size = 6
175
176             # Reset for the next window
177             x_points = []
178             y_points = []
179             timer = 0
180
181
182     finally:
183         GPIO.cleanup()

```

This is the main loop of the program. It continuously polls for data and increments timer. Window_size is the number of data points that are sampled in a frame. Once timer exceeds the window size, the moving average of the data points in that frame is calculated and then the first derivatives are calculated. Eliminate_square_wave is then called and if a square wave is returned then the program will calculate the frequency. If the first time eliminate_square_wave is called, it returns not square wave, then another moving average is done on the first derivatives and the second derivatives are calculated. Since the first derivative of a triangle wave is a square wave, that means the second derivative of the triangle wave will be an impulse function and if the logic of flat regions and abrupt changes is True then the waveform is a triangle wave and then the frequency is then calculated. Otherwise the wave is a sine wave.

At lower frequencies, the window size dynamically adjusts ensure that enough data points are collected to accurately calculate the frequency. In the higher ranges, where the triangle and sine waves appear similar and there is more noise, the `sma_size` is adjusted (smaller to avoid oversmoothing).

Lab Questions

Provide the **questions and answers** to the questions included in the lab manual.

1. Summarize the difference between SPI and I2C ports. Explain in what situation using the SPI ports is better than the I2C ports, and vice versa.
 - a. The difference between the SPI and I2C ports is that the SPI port is a faster bus than the I2C. Conversely, the I2C port only uses two wires to connect many devices, whereas every additional slave device when using the SPI port requires an extra wire. SPI ports should be used in scenarios where speed is the most important factor in the communication and where only master device is used. I2C ports should be used in scenarios where complexity and cost are the most important factor as it is easy to add other devices since only 2 wires are maintained when using the I2C ports.
2. What are the various types of ADCs in use? Which type of ADC is MCP3008 and what are its advantages/disadvantages?
 - a. Various Types of ADCs: SAR (Successive Approximation), Flash, Pipeline, Sigma-Delta. MCP3008 is an SAR ADC. The advantages of a SAR ADC are that it is high performance and low power consumption. The disadvantages of SAR is that when faced with higher resolution needs, since its sampling rate is not enough for those applications.
3. What is the sampling rate for your oscilloscope?
 - a. For lower frequencies, the sampling rate was 20,000 data points per frame and for higher frequencies, 10,000 data points per frame
4. If you use the same Raspberry Pi to do waveform generation and waveform recognition at the same time, you might generate a waveform that the frequency keeps changing and get random readings from the MCP3008. Explain why this is the case.
 - a. This is the case, because of latency in the generation of the waveform to the recognition of the waveform. Because of execution time it is impossible for both to occur at exactly the same which will result in random readings and a frequency that is constantly changing.
5. It is highly likely that your sampled data contains lots of noise. How can you filter the noise? Explain your method.
 - a. We did a simple moving average of the sampled data to filter the noise and adjusted the number of data points in a single window for the moving average depending on the shape of the waveform and the frequency, i.e. a smaller window for the moving average at higher frequencies to avoid oversmoothing and then a larger window at lower frequencies because more data points had to be captured; Therefore, there would need to be more smoothing.

